TP 0 : Les Fonctions

1. Introduction

Nous avons vu un certain nombre de fonctions prédéfinies. Il est possible de définir ses propres fonctions. La première méthode permet de définir des fonctions simples sur une ligne de commande. La seconde, beaucoup plus générale permet de définir des fonctions très évoluées en la définissant dans un fichier.

2. Fonctions inline

Admettons que je veuille définir une nouvelle fonction que j'appelle *sincos* définie mathématiquement par :

$$\text{sincos}(x) = \sin x - x \cos x$$

On ecrira :

```
>> sincos = inline('sin(x)-x*cos(x)')
```

Avouez qu'on peut difficilement faire plus simple ! N'oubliez cependant pas les quotes, qui définissent dans MATLAB des chaines de caractères. On peut maintenant utiliser cette nouvelle fonction :

```
>> sincos(pi/12)
```

Essayons maintenant d'appliquer cette fonction a un tableau de valeurs :

```
>> sincos(0:pi/3:pi)
??? Error using ==> inline/subsref
Error in inline expression ==> sin(x)-x*cos(x)
??? Error using ==> *
Inner matrix dimensions must agree.
```

Ça ne marche pas. Vous avez compris pourquoi ?
MATLAB essaye de multiplier x vecteur ligne par cos(x) aussi vecteur ligne au sens de la multiplication de matrice ! Par conséquent il faut bien utiliser une multiplication terme à terme .* dans la définition de la fonction :

```
>> sincos = inline('sin(x)-x.*cos(x)')
```

et maintenant ,ça marche :

```
>> sincos(0:pi/3:pi)
```

On peut donc énoncer la règle approximative suivante :

> Lorsque l'on définit une fonction, il est préférable d'utiliser systématiquement les opérateurs terme à terme .* ./ et .^ au lieu de * / et ^ si l'on veut que cette fonction puisse s'appliquer à des tableaux.

On aura remarqué que la commande inline reconnait automatiquement que la variable est x.
On peut de même définir des fonctions de plusieurs variables :

```
>> sincos = inline('sin(x)-y.*cos(x)')
sincos =
     Inline function:
     sincos(x,y) = sin(x)-y.*cos(x)
```


Scanné avec CamScanner

Ici encore, MATLAB a reconnu que les deux variables étaient x et y. L'ordre des variables (affiché à l'écran) est celui dans lequel elles apparaissent dans la définition de la fonction.

3. Fonctions définies dans un fichier

Pour définir une nouvelle fonction sous Matlab, on écrit la définition de la fonction dans un fichier avec une extension .m (fonction M-File). Le nom du fichier doit être le nom de la première fonction définie (la seule visible). On l'exécute en tapant le nom de la fonction avec entre parenthèses la liste des arguments.

La première ligne du fichier de la fonction doit suivre la syntaxe suivante:

function arguments de sortie = **nom**(arguments d'entrée)

On doit donc déclarer les valeurs calculées par la fonction dans les arguments de sortie, et les paramètres de la fonction dans les arguments d'entrée (qui ne sont pas modifiés).

Attention, contrairement au script, les variables d'une fonction sont des variables locales, qui ne sont pas accessibles en dehors de la fonction. Pour passer des informations à une fonction, il faut donc utiliser les paramètres d'entrée ou de sortie. En fait on peut utiliser des variables globales (mot clé global) sous Matlab, mais cela est à éviter dans le cadre d'une bonne programmation.

C'est la méthode la plus généraliste, et elle permet notamment de réaliser des fonctions ayant plusieurs sorties. Commençons par reprendre l'exemple précèdent (sincos). L'ordre des opérations est le suivant :

1. Editer un nouveau fichier appelé sincos.m
2. Taper les lignes suivantes :
   ```
   function s = sincos(x)
   s = sin(x)-x.*cos(x);
   ```
3. Sauvegardez

Le résultat est le même que précédemment :

```
>> sincos(pi/12)
ans =
0.0059
```

On remarquera plusieurs choses :

- l'utilisation de .* pour que la fonction soit applicable à des tableaux.
- la variable s n'est là que pour spécifier la sortie de la fonction.
- l'emploi de ; à la fin de la ligne de calcul, sans quoi MATLAB afficherait deux fois le résultat de la fonction.

Donc encore une règle :

> Lorsque l'on définit une fonction dans un fichier, il est préférable de mettre un ; à la fin de chaque commande constituant la fonction. Attention cependant à ne pas en mettre sur la première ligne.

Un autre point important méritant un cadre :

> Le nom du fichier doit porter l'extension .m et le nom du fichier sans suffixe doit être exactement le nom de la fonction (apparaissant après le mot-clé **function**)

Voyons maintenant comment définir une fonction comportant plusieurs sorties. On veut réaliser une fonction appelée **cart2pol** qui convertit des coordonnées cartésiennes (x, y) (entrées de la fonction) en coordonnées polaires (r, θ) (sorties de la fonction). Voilà le contenu du fichier **cart2pol.m** :

Scanné avec CamScanner

```
function [r, theta] = cart2pol (x, y)
r = sqrt(x.^2 + y.^2);
theta = atan (y./x);
```

(atan : inverse de tang)

On remarque que les deux variables de sortie sont mises entre crochets, et séparées par une virgule.

Pour utiliser cette fonction, on écrira par exemple :

```
>> [rr,tt] = cart2pol(1,1)
rr =
    1.4142
tt =
    0.7854
```

On affecte donc simultanément deux variables **rr** et **tt** avec les deux sorties de la fonction, en mettant ces deux variables dans des crochets, et séparés par une virgule. Les crochets ici n'ont bien sûr pas le même sens que pour les tableaux.

Il est possible de ne récupérer que la première sortie de la fonction. MATLAB utilise souvent ce principe pour définir des fonctions ayant une sortie «principale» et des sorties «optionnelles».

Ainsi, pour notre fonction, si une seule variable de sortie est spécifiée, seule la valeur du rayon polaire est renvoyée. Si la fonction est appelée sans variable de sortie, c'est **ans** qui prend la valeur du rayon polaire :

```
>> cart2pol(1,1)
ans =
    1.4142
```

La fonction **racine** ci-dessous est écrite dans un fichier **racine.m**.

```
function x=racine(a,b)
% calcul la racine de f(x) définie ci-après sur [a,b]
fa=f(a); fb=f(b); x=(a+b)/2;
if (fa*fb>0) x=-Inf; return; end;
while (b-a)>eps*x
 x=(a+b)/2; fx=f(x);
 if(sign(fx)==sign(fa))
  a=x; fa=fx;
 else
  b=x; fb=fx;
 end;
end;
% definition de f(x) (fonction locale)
function y=f(x)
y=x^3-2*x-5;
```

Pour l'exécuter, on tape simplement

```
>> racine(0,3)
ans =
    2.0946
```

Scanné avec CamScanner

Exercice 1

Écrire un .m file pour la fonction $f_1(x) = \frac{x^5 - 3}{\sqrt{x^2+1}}$

1. Tester la fonction sur quelques valeurs, par exemple vérifier que : $f_1(1) = -1.4142$, $f_1(0) = -3$
2. Créer un tableau x d'abscisses de -5 à 5 par pas de 0.1 3
3. Représenter graphiquement la fonction f_1 aux points x_1.

Exercice 2

Soient la matrice et les vecteurs colonnes suivants

$$A = \begin{pmatrix} 5/8 & -1/4 & 1/8 \\ 1/4 & 0 & 1/4 \\ 1/8 & -1/4 & 5/8 \end{pmatrix}, \vec{b} = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}, \vec{u}_1 = \begin{pmatrix} 5 \\ 2 \\ -4 \end{pmatrix}.$$

On définit, pour $n \geq 1$, la suite de vecteurs $\vec{u}_{n+1} = A\vec{u}_n + \vec{b}$.

Construire une fonction suite.m calculant les premiers termes de la suite $\vec{u}_n$. Cette fonction aura comme arguments d'entrée les données suivantes : la matrice A, le second membre $\vec{b}$, le terme initial $\vec{u}_1$, et le nombre de termes voulus nb_{it}.

Exercice 3

Construire une fonction **grand.m** qui permet de trouver le plus grand élément dans un vecteur de nombres.

Scanné avec CamScanner

Polynômes et interpolation polynomiale

1. Opérations sur les polynômes dans MATLAB

MATLAB représente les polynômes sous forme de vecteurs lignes dont les composantes sont ordonnées par ordre des puissances décroissantes. Un polynôme de degré n est représenté par un vecteur de taille (n+1).

$$f(x) = 3x^6 - 2x^4 + x^3 + 5x^2 - x + 9$$

Il est représenté par le vecteur ligne :

```
>> f=[3  0  -2  1  5  -1  9]
```

En plus des opérations propres aux vecteurs qui peuvent être utilisées pour la manipulation des polynômes, d'autres opérations spécifiques sont prévues dans MATLAB, telles que conv, deconv, roots, etc.

2. Multiplication, division et racines de polynômes

La multiplication et la division de deux polynômes sont réalisées respectivement par la convolution et la déconvolution de leurs coefficients, à l'aide des fonctions conv et deconv.

Considérons le système suivant constitué de deux fonctions de transfert mises en cascade.



Avec:

$$H_1(p) = \frac{p+1}{2p^2+p+2}$$

$$H_2(p) = \frac{2p+1}{p^2+2}$$

La fonction de transfert globale est le produit des fonctions de transfert élémentaires :

$$H(p)=H_1(p)\,H_2(p)$$

Fonctions de transfert individuelles :

```
>> numerateur_H1=[1,1]
>> denominateur_H1=[2,1,2]
>> numerateur_H2=[2,1]
>> denominateur_H2=[1,0,2]
>>
```

La multiplication de polynômes se fait grâce à la commande conv. Donc la fonction de transfert globale H est donnée par :

Scanné avec CamScanner

```
>> numerateur_H=conv(numerateur_H1,numerateur_H2)
>>denominateur_H=conv(denominateur_H1,denominateur_H2)
>>
```

Le même résultat peut être obtenu en composant plusieurs fois la fonction conv.

```
>>num_H=conv([1,1],[2,1])
>>deno_H=conv([2,1,2],[1,0,2])
>>
```

Pour afficher cette fonction de transfert globale sous forme d'une fraction, nous pouvons utiliser la fonction printsys de la boîte à outils "Control System Toolbox".

```
>>printsys(num_H,deno_H,'p')
>>
```

La division de polynômes se fait grâce à la commande deconv. Elle donne le rapport de convolution de deux polynômes (déconvolution des coefficients du polynôme). L'exemple suivant montre l'utilisation de cette fonction. Soit les deux polynômes :

$$Y = 2x^3 + 6x^2 + 10x + 12$$

$$Z = 6x^2 + 5x + 6$$

Pour la division, si Y n'est pas un facteur de Z, on obtiendra un "reste" après la division.

```
>> Y=[2 6 10 12];Z=[6 5 6];
>> deconv(Y,Z)
>>
```

Les racines d'un polynôme sont données par la fonction roots. Ceci est intéressant en traitement de signal ; les solutions du numérateur (zéros) et ceux du dénominateur (pôles) de la fonction de transfert peuvent renseigner sur les modes du système.

Considérons la fonction de transfert *H(p)* précédente.

```
>>zeros_H = roots(num_H)
>>poles_H = roots(deno_H
>>
```

Inversement, on peut construire un polynôme à partir de ses racines en utilisant la fonction poly. Les racines du polynôme sont passées à la fonction sous forme d'un vecteur.

Scanné avec CamScanner

```
>>num_H = poly(zeros_H)
>>den_H = poly(poles_H)
>>
```

La fonction poly accepte aussi une matrice comme argument dont elle retourne le polynôme caractéristique.

```
>> A = [2 0 4; 1 5 3; 2 4 1];
>> P = poly(A)
>> B = [2 0 4; 1 5 3];
>> poly(B) % la matrice doit être carrée
>>
```

Le polynôme caractéristique de la matrice A est alors :

$$P = x^3 - 8x^2 - 3x + 38$$

Les racines de ce polynôme sont les valeurs propres de la matrice A que l'on peut aussi obtenir directement par la fonction eig.

```
>> racines = roots(P)
>>val_prop = eig(A)
>>
```

Pour évaluer le polynôme P(x)en un point donné, on doit utiliser la fonction polyval . On évalue ce polynôme pour x=donnée, par exemple : On veut évaluer en $x=2$ le polynôme suivant :

$$T = 3x^4 + x^3 + 2x^2 - 3x + 1$$

```
>> T=[3 1 2 -3 1];
>> x=2;
>>y =polyval(T,x)
>>
```

Si x est un vecteur contenant plusieurs valeurs, y sera aussi un vecteur qui contiendra le même nombre d'éléments que x.

```
>> x=2 :1 :4
>>y =polyval(T,x)
```


Scanné avec CamScanner

3- Interpolation polynomiale

Dans le domaine de l'analyse de données, on a souvent besoin d'établir un modèle mathématique liant plusieurs séries de valeurs expérimentales. L'interpolation consiste à approcher la courbe liant deux séries de mesures par un polynôme. Les coefficients optimaux de ce polynôme sont ceux qui minimisent la variance de l'erreur d'interpolation. Ce principe est connu sous le nom de la méthode des moindres carrés. La commande p = polyfit(x,y,n) retourne le polynôme p de degré n permettant d'approcher la courbe y = f(x) au sens des moindres carrés. Afin d'en déduire l'erreur entre la courbe expérimentale et le modèle obtenu, on dispose de la fonction polyval(p,x) qui retourne la valeur du polynôme p pour toutes les composantes du vecteur ou de la matrice x. Pour appliquer ces fonctions, nous allons simuler une courbe expérimentale par une sigmoïde à laquelle nous superposons un bruit gaussien.

fichier sig_reg.m

```
% génération de la courbe expérimentale
% effacement des variables de l'espace de travail
% intervalle de définition et calcul de la sigmoïde
x = -5:0.1:5;
% fonction sigmoïde bruitée
y = 1./(1+exp(-x))+ 0.05*randn(1,length(x));
% tracé de la sigmoïde bruitée
plot(x,y)
title('fonction sigmoïde bruitée, polynôme d''interpolation')
% polynôme d'ordre 1 d'interpolation
p = polyfit(x,y,1);
% valeurs du polynôme d'interpolation
polyn = polyval(p,x); hold on
% tracé du polynôme d'interpolation
plot(x,polyn,'-.')
% calcul de l'erreur d'interpolation
err = y - polyn;
% tracé de la courbe de l'erreur
plot(x,err,':'), grid
gtext('mesures'), gtext('erreur'), gtext('modele'), hold off
% affichage du polynôme d'interpolation
disp('polynôme d''interpolation')

var_err = num2str(std(err)^2);
disp(['variance de l''erreur d''interpolation : ' ,var_err])
```

```
>> sig_reg
```

On remarque ici que le polynôme d'interpolation d'ordre 1 n'est pas une bonne approximation pour ces données. En effet, un polynôme d'ordre 5 donne une meilleure approximation de la sigmoïde (fig. 2). Ainsi, dans le programme précédent (**sig_reg.m**), on change dans le '1' par '5' dans la fonction '**polyfit**'.

Scanné avec CamScanner

```
>> sig_reg
```

Une interpolation consiste à relier les points expérimentaux par une courbe sous forme de segments de droites ou de courbes polynomiales. Ceci peut être réalisé par la fonction 'interp1'. La commande 'interp1(x,y,xi,'type')' retourne un vecteur de mêmes dimensions que xi et dont les valeurs correspondent aux images des éléments de xi déterminées par interpolation sur x et y. Si f est l'interpolation de y, la chaine 'type' spécifie alors le type d'interpolation qui doit être parmi les suivants :

- 'linear' ▫ : interpolation linéaire
- 'spline' ▫ : interpolation par splines cubiques,
- 'cubic' ▫ : interpolation cubique.

Si on ne spécifie pas le type, l'interpolation linéaire est choisie par défaut.

```
>> x = 0:10; y = sin(x); xi = 0:.25:10;
>> yi = interp1(x,y,xi)
>> plot(x,y,'o',xi,yi)
>> hold on
>> yi = SPLINE(x,y,xi)
>> plot(x,y,'o',xi,yi)
```

La fonction interpolLavec.m qui suit calcule le polynôme d'interpolation de Lagrange pour la fonction $f(x) = 1/(1+x^2)$; les résultats sont obtenus avec le script interpolLavec_T.m

```
>> interpolLavec_T
```


Scanné avec CamScanner